Lecture 9: Scaling Laws

Scaling the World

Imagine the following scenario..

Your friend has given you ten thousand B200s for a month, and asked you to build a good open source LM.

What do you do?

- Put together your infra team and distributed training framework (A2).
- Assemble your great pretraining dataset (A4).
- Run a big model (but which one??) ⟵ we are here.

Wide or deep? How many heads? Which nonlinearity?

Name	Has paper	Link	Year	Tokenizer type	Vocab count	Norm	Parallel Layer	Pre-norm	Position embedding	Activations	MoE	MLP factor	num_layers	model_dim
Original transformer	Yes	arxiv.org/abs...03762	2017	BPE	37000	LayerNorm	Serial	☐	Sine	ReLU	☐	4	6	
GPT	Yes	cdn.openai.com/res...er.pdf	2018	BPE	40257	LayerNorm	Serial	☐	Absolute	GeLU	☐	4	12	
GPT2	Yes	cdn.openai.com/bet...rs.pdf	2019	BPE	50257	LayerNorm	Serial	☑	Sine	GeLU	☐	4	48	
T5 (11B)	Yes	arxiv.org/abs...10683	2019	SentencePiece	32128	RMSNorm	Serial	☑	Relative	ReLU	☐	64	24	
GPT3 (175B)	Yes	arxiv.org/abs...14165	2020	BPE	50257	LayerNorm	Serial	☑	Sine	GeLU	☐	4	96	
mT5	Yes	arxiv.org/abs...11934	2020	SentencePiece	250000	RMSNorm	Serial	☑	Relative	GeGLU	☐	2.5	24	
T5 (XXL 11B) v1.1	Kind of	github.com/goo...d#t511	2020	SentencePiece	32128	RMSNorm	Serial	☑	Relative	GeGLU	☐	2.5	24	
Gopher (280B)	Yes	arxiv.org/abs...11446	2021	SentencePiece	32000	RMSNorm	Serial	☑	Relative	ReLU	☐	4	80	
Anthropic LM (not claude)	Yes	arxiv.org/abs...00861	2021	BPE	65536			☑			☐	4	64	
LaMDA	Yes	arxiv.org/abs...08239	2021	BPE	32000			☑	Relative	GeGLU	☐	8	64	
GPTJ	Kind of	huggingface.co/Ele...t-j-6b	2021	BPE	50257	LayerNorm	Parallel	☑	RoPE	GeLU	☐		28	
Chinchilla	Yes	arxiv.org/abs...15556	2022	SentencePiece	32000	RMSNorm	Serial	☑	Relative	ReLU	☐	4	80	
PaLM (540B)	Yes	arxiv.org/abs...02311	2022	SentencePiece	256000	RMSNorm	Parallel	☑	RoPE	SwiGLU	☐	4	118	
OPT (175B)	Yes	arxiv.org/abs...01068	2022	BPE	50272	LayerNorm	Serial	☑	Absolute	ReLU	☐	4	96	
BLOOM (175B)	Yes	arxiv.org/abs...05100	2022	BPE	250680	LayerNorm	Serial	☑	AliBi	GeLU	☐	4	70	
GPT-NeoX	Yes	arxiv.org/pdf...45.pdf	2022	BPE	50257	LayerNorm	Parallel	☑	RoPE	GeLU	☐	4	44	
GPT4	Ad	arxiv.org/abs...08774	2023	BPE	100000			☐			☐			
LLaMA (65B)	Yes	arxiv.org/abs...13971	2023	BPE	32000	RMSNorm	Serial	☑	RoPE	SwiGLU	☐	2.6875	80	
LLaMA2 (70B)	Yes	arxiv.org/abs...09288	2023	BPE	32000	RMSNorm	Serial	☑	RoPE	SwiGLU	☐	3.5	80	
Mistral (7B)	Yes	arxiv.org/abs...06825	2023	BPE	32000	RMSNorm	Serial	☑	RoPE	SwiGLU	☐	3.5	32	

We could cargo cult things from existing LMs .. but how do these get optimized in the first place?

Today: simple, predictive ”laws” for behaviors of LMs

The approach -

scaling laws which are simple, predictive rules for model performance.

The experiment -

- **Old and unpleasant**: tune hyperparameters on big models.
- **New (over?) optimism**: tune on small models, extrapolate to large ones.

Sample complexity and rates

Theorists have thought about "scaling" for a long time ..

$$\epsilon(\hat{h}) \leq \min_{h \in H} \epsilon(h) + 2\sqrt{\frac{1}{m} \log \frac{2k}{\delta}}$$

(learning in a finite set of k hypotheses)

Under the assumptions of Theorem 1.5 (somewhere), the rate of convergence of the estimator $\hat{p}_n(x_0)$ is $\psi_n = n^{-\frac{\beta}{2\beta+1}}$, which means that for a finite constant C and for all $n \geq 1$ we have

$$\sup_{p \in \mathcal{P}(\beta, L)} \mathbb{E}_p \left[(\hat{p}_n(x_0) - p(x_0))^2 \right] \leq C\psi_n^2$$

(generative modeling for smooth densities)

But these are upper bounds, not actual realized loss values.

Hestness et al 2017

Earliest "large scale network" scaling work: Hestness 2017.

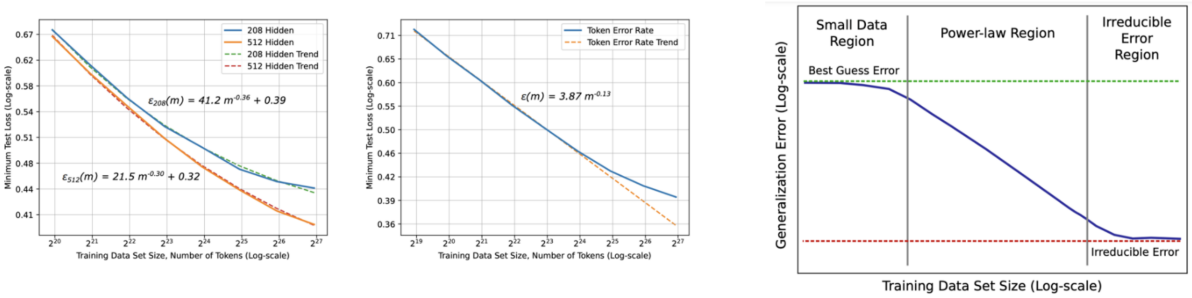


Figure 1: Neural machine translation learning curves. Left: the learning curves for separate models follow $\varepsilon(m) = \alpha m^{\beta_g} + \gamma$. Right: composite learning curve of best-fit model at each data set size.

Predictable scaling on many tasks (MT, LM, Speech) and hypothesized scaling shape.

Hestness II

Very ahead of its time ..

"Emergence"

Although small data set testing may be possible, it can be difficult to ensure that training data is large enough to see that power-law learning curve region. We have found that models with poor optimizer parameterization or model priors / initialization how accuracy cliffs, where accuracy is only as good as best guessing, but the model trains on enough data to be in the lower-law region. Researchers must take great care when defining a "large enough" training set for small data testing. We leave the methodology for defining such a training set to future work.

Scaling by computing

Computational limits: If we have identified a desirable model to scale to larger training sets, the next potential limitation is the speed of computation. In some cases, training large models on very large data sets would take months or years of critical path compute time, making these training runs impractical for any real world problem on existing systems. However, predictable learning and model size curves may offer a way to project the compute requirements to reach a particular accuracy level. The compute requirements could inform decision about how to scale computational capacity to unlock these compute-limited applications.

Speed = accuracy

The Performance-Accuracy Trade-off: Many DL software and hardware techniques impose a trade-off between model accuracy and the speed of computation. Learning curves and model size growth can indicate whether these techniques could regain lost accuracy by improving the speed of computation. For example, low-precision computation/quantization and sparse models give up som model accuracy (e.g. up to 20%) in order to improve compute throughput. If the compute throughput improvements allow DL engineers to train larger models on larger data sets, these accuracy losses might be easily recoverable.

Neural (LLM) scaling behaviors

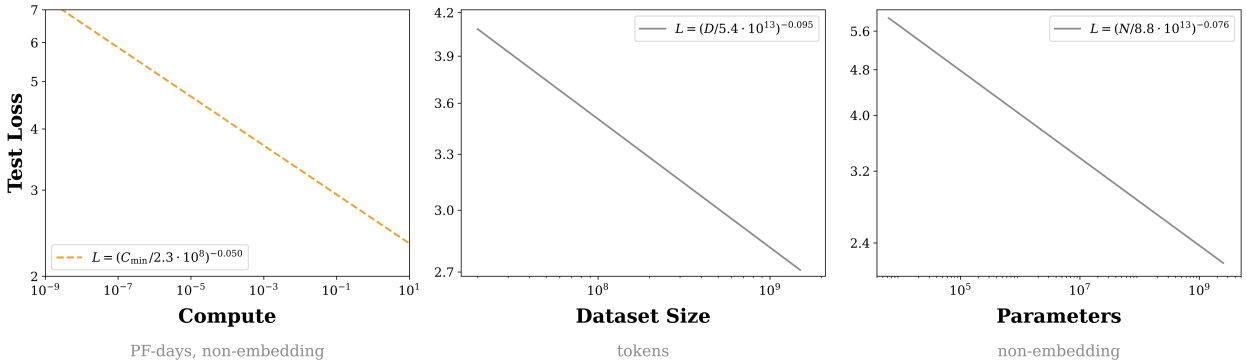
1. Data vs performance: ”Are there simple rules that determine how data affects performance?”

2. Data vs model size: ”Do we train on more data or bigger models?”

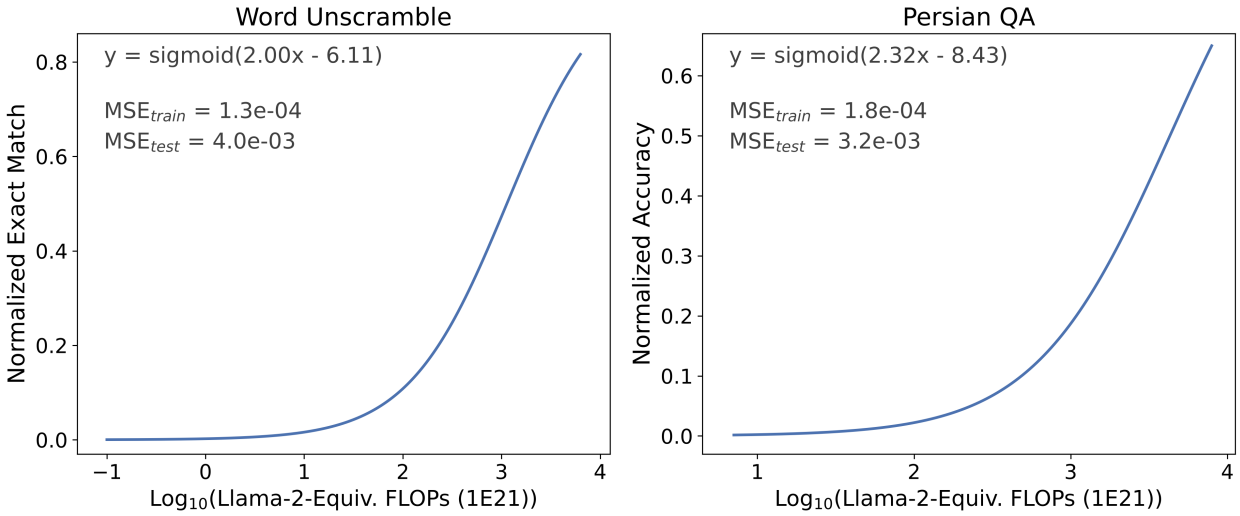
3. Parameters vs performance: ”How should we set the hyperparameters on the big model??”

Scaling laws - power law relationships for many factors

These scaling laws hold on *many* different kind of phenomena!



They even hold in nonstandard settings.



Data vs performance

What's a data scaling law?

Data scaling laws : simple formula that maps dataset size (n) to error!

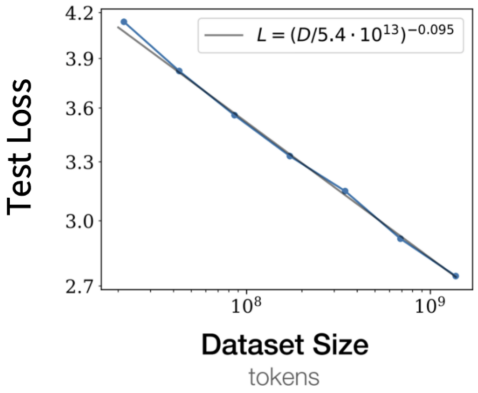
What do we expect out of scaling laws?

Monotonic, logistic(-like) curves!

Data scaling laws for language models

First, an empirical observation!

Loss and dataset size is linear on a log-log plot



"Scale-free" or "Power law"

(For language modeling, from Kaplan+ 2020)

Conceptual foundations of data scaling laws

Question: *Why do scaling laws show up?*

We know error should be monotone, but why is it a power law / linear in log-log?

Claim: *Estimation error naturally decays polynomially.*

But this answer may take a moment to understand. Let's work through an example.

Example: *If our task is to estimate the mean of a dataset, what's the scaling law?*

Toy example: mean estimation

Input: $x_1 \dots x_n \sim N(\mu, \sigma^2)$

Task: estimate the average as $\hat{\mu} = \frac{\sum_i x_i}{n}$

What's the error? By standard arguments ..

$$\mathbb{E}[(\hat{\mu} - \mu)^2] = \frac{\sigma^2}{n}$$

This is a "scaling law"

$$\log(\text{Error}) = -\log n + 2\log \sigma$$

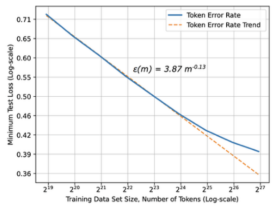
More generally, any polynomial rate $1/n^\alpha$ is a scaling law.

Scaling law exponents: an intriguing mystery

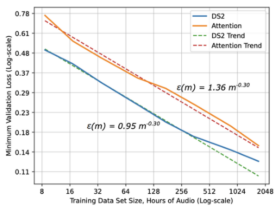
Fact: *Similar arguments show most "classical" models (regression, etc) have* $\frac{1}{n}$ *scaling.*

This means we should see $y = -x + C$.

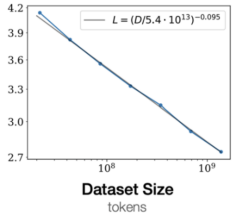
Question: *What do we find in neural scaling laws?*



Machine translation



Speech



Language modeling

Very different from predictions .. Why might this be?

Detour: scaling laws for (nonparameteric) learning

Neural networks can approximate arbitrary functions. Lets turn that into an example.

Input: $x_1 \dots x_n$ uniform in 2D unit box. $y_i = f(x_i) + N(0, 1)$!

Task: estimate $f(x)$.

Approach: cut up the 2D space into boxes with length of $n^{-\frac{1}{4}}$!

What's our estimation error?

Informally, we have $\sqrt{n}$ boxes, each box gets $\sqrt{n}$ samples.

$$\text{Error} \approx \frac{1}{\sqrt{n}} + (\text{other smoothness terms})$$

In d-dimensions, this becomes Error $= n^{-\frac{1}{d}}$. **This means scaling is** $y = -\frac{1}{d}x + C$**.**

Claim: *Flexible "nonparametric" learning has dimension dependent scaling laws.*

Intrinsic dimensionality theory of data scaling laws

Some have made the following argument (Bahri 2021)

1. Scaling laws arise due to polynomial rates of learning $\frac{1}{n^\alpha}$.
2. Some argue the slope α is closely connected to the *intrinsic dimensionality* of the data.

But estimators of intrinsic dimension are sketchy, and this is not airtight ..

Other data scaling laws

Data scaling thus far: how does dataset size relate to performance?

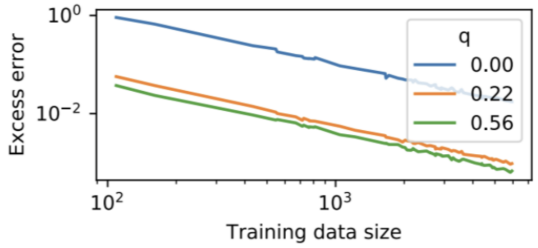
- Picking optimal data mixture using small scale models
- Deciding whether to repeat data or not
- Combining the two and balancing quality with repetition rate

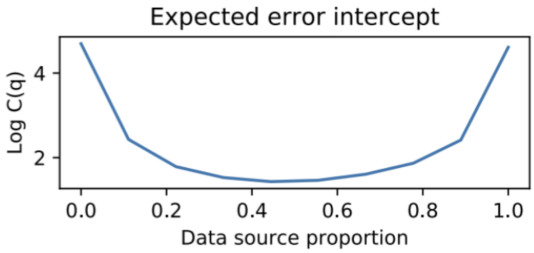
Related question: how does dataset *composition* affect performance?

Other adavanced data scaling law: distribution shift

Claim: *Data composition affects the offset, not the slope.*

These "distribution shift" scaling laws can tell us about the importance of collecting diverse data!





In practice: data mixture selection via scaling is hard

Natural idea - build data scaling laws

Empirical eval - just take the best small dataset

Scaling laws under data repetition

In practice, we have finite data - how does repeating examples affect scaling?

$$D' = U_D + U_D R_D^*(1 - e^{-\frac{R_D}{R_D^*}}).$$

Scaling laws in *compute unbounded* settings

1. Scaling laws can "break" if you blindly apply it
2. Scaling laws are *lower bounds* so you can always potentially do better

Data selection scaling and accounting for finiteness

Given that repeated data is less valuable .. Data selection should then be adaptive to scale!

Scaling laws for model engineering

Now for what I promised at the start: **model scaling!**

Question: *How can we efficiently design huge LMs?*

- LSTMs vs Transformers
- SGD vs Adam

Question: *How should we allocate our limited resources?*

- Train models longer vs train bigger models?
- Collect more data vs get more GPUs?

Scaling laws provide a simple procedure to answer these questions.

Hyperparameter questions

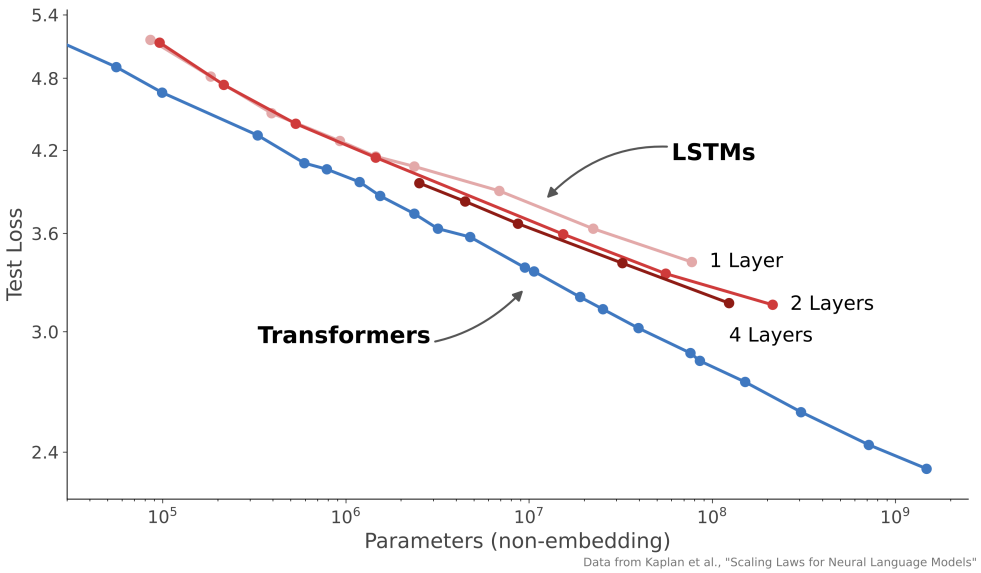
We'll consider some of these choices in the context of the classic Kaplan scaling paper!

- Architecture
- Optimizer
- Aspect ratio / depth
- Batch size

1. Architecture: Transformers vs LSTMs

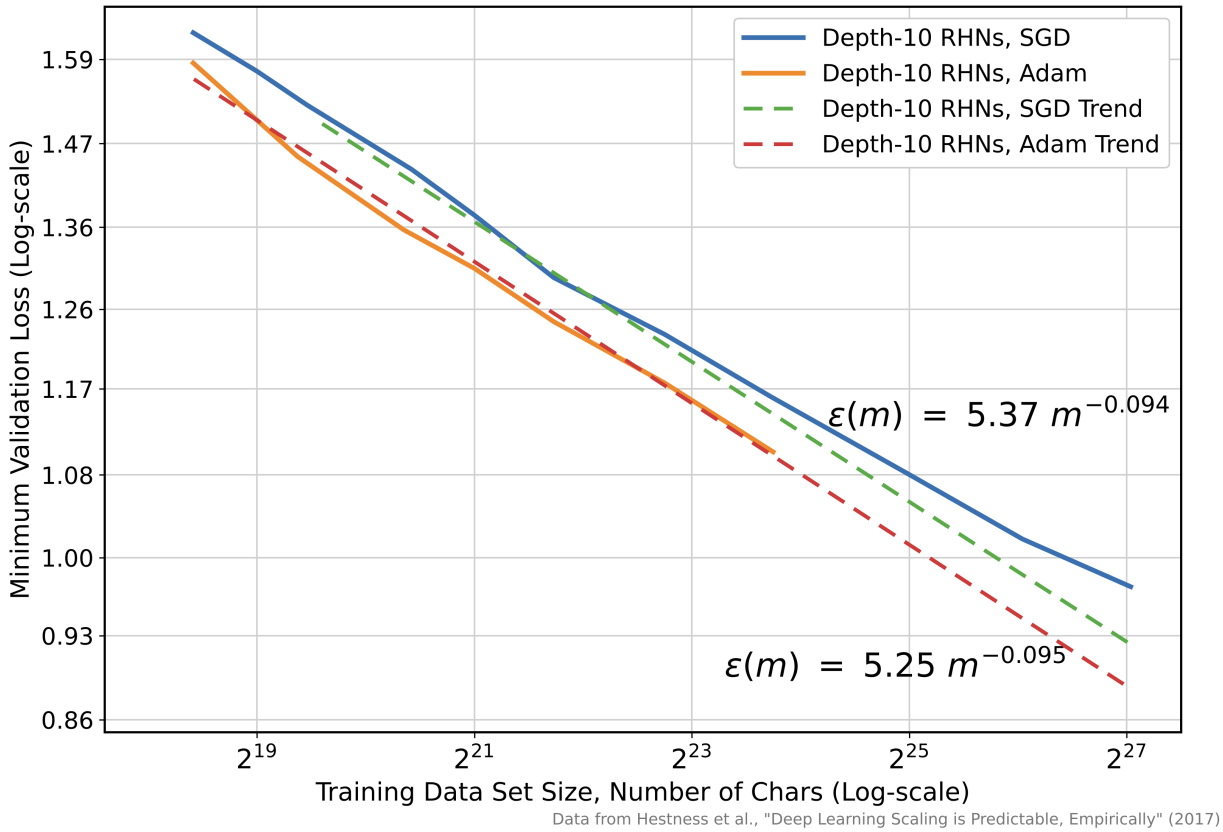
Question: *Are transformers better than LSTMs?*

- Brute force way: spend tens of millions to train a LSTM GPT-3
- Scaling law way:



2. Optimizer choice: Adam vs SGD

Question: *What about ADAM vs SGD?*



(Note, this is in 2017, so pre-transformers. RHN are recurrent highway nets)

3. Depth/width: Number of layers

Question: *Does the depth or width make a huge difference?*

- 1 vs 2 layers makes a huge difference.
- More layers have diminishing returns below 10^7 params

3. Depth/width: Other Transformer hypers

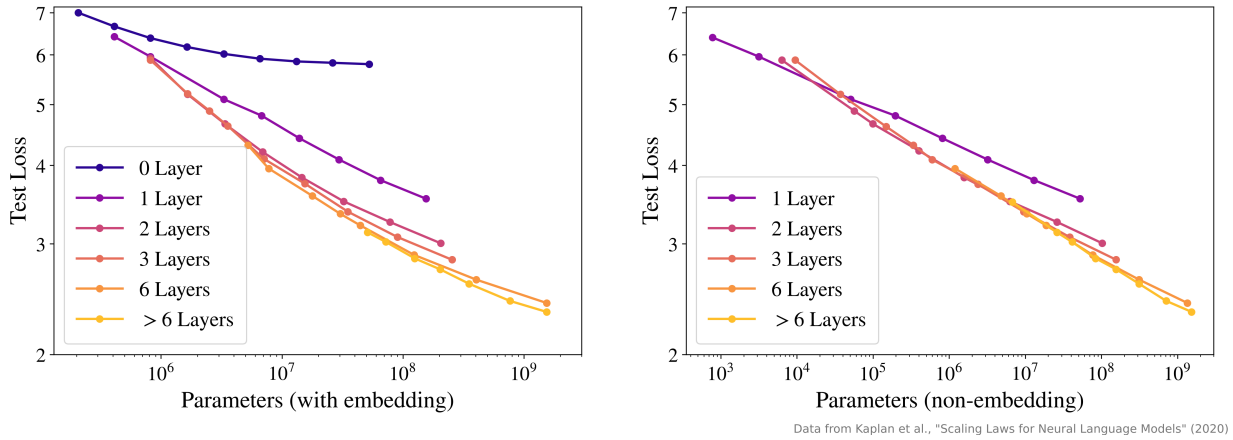
Question: *Do hyperparameters like the aspect ratio depend on scale?*

Performance depends very mildly on model shape when the total number of non-embedding parameters N is held fixed. The loss varies only a few percent over a wide range of shapes. Small

differences in parameter counts are compensated for by using the fit to $L(N)$ as a baseline. Aspect ratio in particular can vary by a factor of 40 while only slightly impacting performance; an $(n_{\text{layer}}, d_{\text{model}}) = (6, 4288)$ reaches a loss witin 3% of the $(48, 1600)$ model used in [RWC+19].

3. Depth/width: But not all parameters are made equal

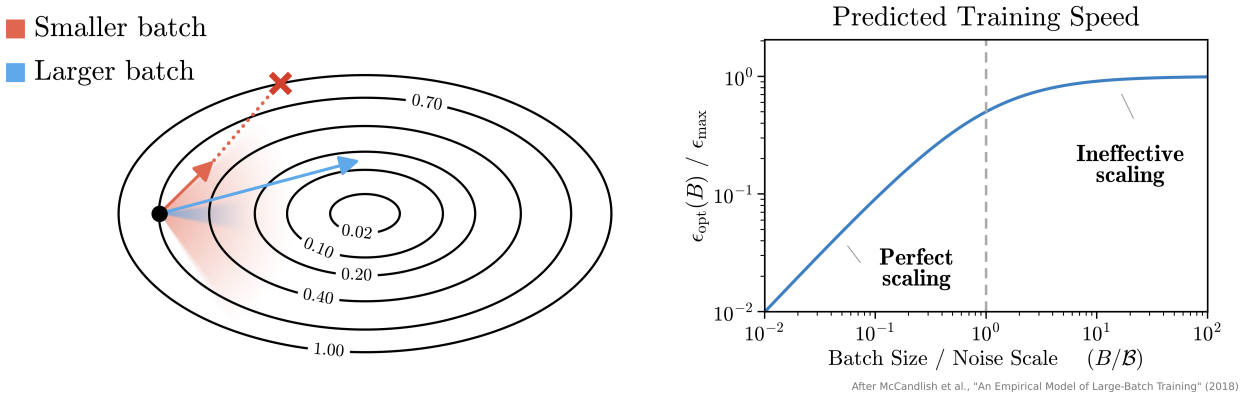
We've been thinking about "parameters" but not all parameters are equal!



- Embedding layer parameters don't behave the same!
- **Related**: recent papers on scaling laws for mixtures of experts.

Remark: *With MoEs, we expect parameter scaling to change.*

4. Batch size: Critical batch size



- **Batch size** - known to have strong diminishing returns past a certain point.

- **Critical batch** = min number of examples before before diminishing returns (what does that mean?)

4. Critical batch size definitions..

Question: *What is critical batch size, more precisely?*

- Pick a target loss and train model (S steps and E examples are needed).
- Sweep over the batches.

The curve should follow roughly ..

$$\frac{S}{S_{\text{min}}} - 1 = \left(\frac{E}{E_{\text{min}}} - 1\right)^{-1}.$$

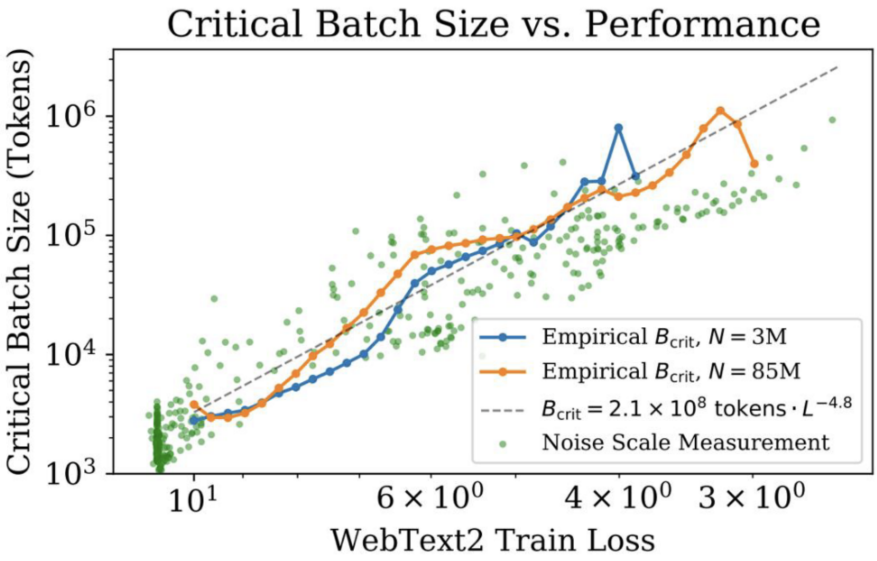
Fit for $S_{\text{min}}, E_{\text{min}}$ on this curve and pick

$$\mathcal{B}_{\text{crit}} = \frac{E_{\text{min}}}{S_{\text{min}}}.$$

This balances both sides of the equation, giving roughly $2\times$ the steps / passes optimal!

Claim: *This is claimed to be close to the ratio of the trace of the gradient covariance and squared norm of the gradient.*

4. Batch size: critical batch size

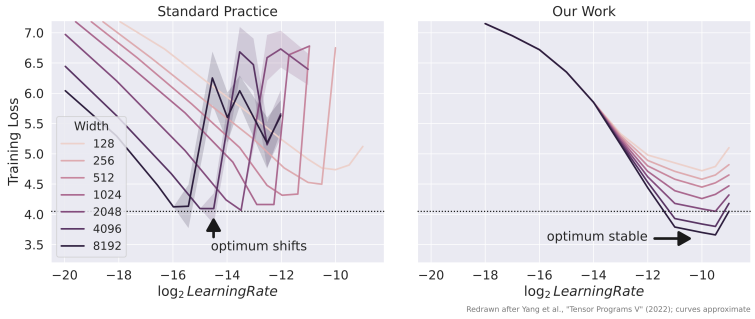


The smaller the loss target, the bigger the batch size

$$C_{\min}(C) \equiv \frac{C}{1 + B/B_{\text{crit}}(L)}.$$

(minimum compute, at $B \ll B_{\text{crit}}$)

5. Learning rates: muP and scale-aware LR choices



μP scaling rules for a model M' that is r times wider than M

Hyperparameter (weight)	M	$M' \sim r$
AdamW learning rate (matrix-like)	l	l/r
AdamW learning rate (others)	l	l
Initialization variance (matrix-like)	σ	σ/r
Initialization variance (others)	σ	σ
Multiplier (output)	τ	τ/r
Multiplier (others)	τ	τ

Matrix-like: a weight with two dimensions that grow with width (e.g. hidden fully-connected layers).
Others: weights with one or no such dimension, including embeddings.
Output: the layer mapping the width to a fixed size, i.e. the decoding layer (lm_head).
Multiplier: a constant scale applied to a weight, playing a role like softmax temperature.

After Yao et al. (2024)

If we naively scale up - optimal learning rate depends on scale.

We need *scaling aware* initialization and learning rate scaling

Caution - scaling behaviors can differ downstream

- **Thus far**: scaling is predictable and depends mainly on parameters.
- **The catch**: downstream scaling can often be much less predictable.

Some surprising takeaways

The effect of hyperparameters on LLMs can be predicted *before* training!

- Optimizer choice

- Model depth

- Architecture choice

The scaling law based design procedure.

1. Train a few smaller models
2. Establish a scaling law (e.g. ADAM vs SGD scaling law)
3. Select optimal hyperparam based on the scaling law prediction

One important use of scaling laws

Question: *Do we need more data or bigger models?*

Clearly, lots of data is wasted on small models.

Joint data-model scaling laws describe how the two variables relate.

From Rosenfeld+ 2020,

$$\text{Error} = n^{-\alpha} + m^{-\beta} + C.$$

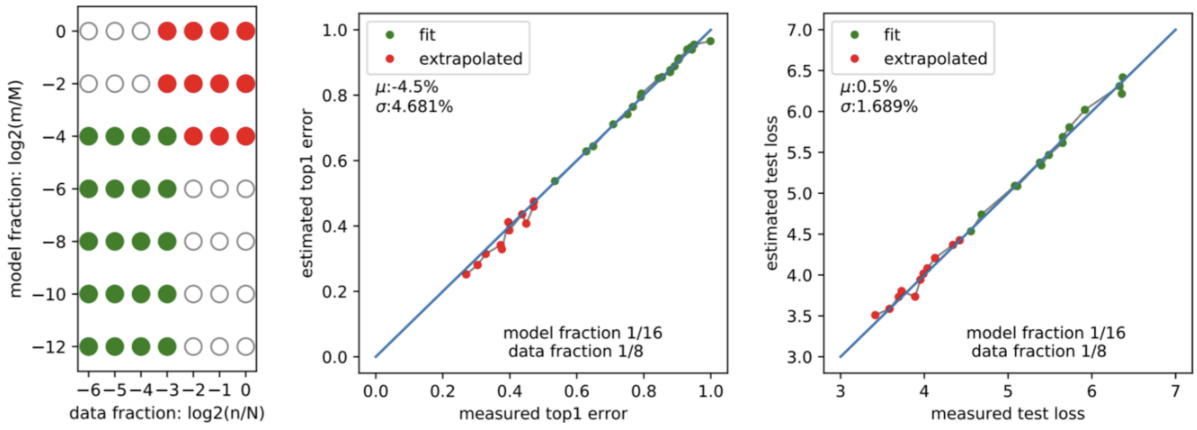
From Kaplan+ 2020,

$$\text{Error} = [m^{-\alpha} + n^{-1}]^{\beta}.$$

Provides surprisingly good fits to model-data joint error.

Model-data joint scaling is accurate

From Rosenfeld - fit scaling exponents on small data, small models. Predict rest on large data, large models.



(a) Illustration. (b) Extrapolation on ImageNet (c) Extrapolation on WikiText-103.

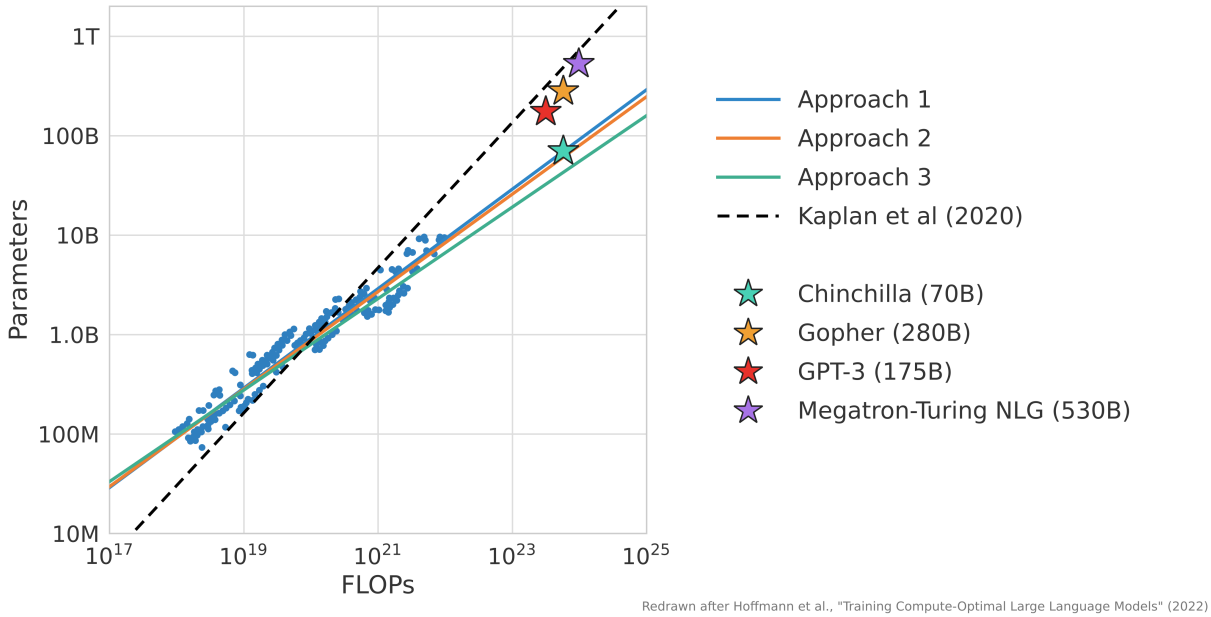
Trading off data and model sizes: optimize $n^{-\alpha} + m^{-\beta} + C$ with your costs.

"Optimal" compute and data tradeoffs as a case study

Rosenfeld, Kaplan both predict relationship of data, model and performance.

Claim (Kaplan): $N_{opt} = C^{0.73}, D_{opt} = C^{0.27}$ *(tokens per param decreases with C).*

Chinchilla [Hoffman et al] argue these fits are quite off.



Redrawn after Hoffmann et al., "Training Compute-Optimal Large Language Models" (2022)

Chinchilla in depth - 3 methods

Approach	Coeff. a where $N_{opt} \propto C^a$	Coeff. b where $D_{opt} \propto C^b$
1. Minimum over training curves	0.50 (0.488, 0.502)	0.50 (0.501, 0.512)
2. IsoFLOP profiles	0.49 (0.462, 0.534)	0.51 (0.483, 0.529)
3. Parametric modelling of the loss	0.46 (0.454, 0.455)	0.54 (0.542, 0.543)
Kaplan et al. (2020)	0.73	0.27

The chinchilla authors suggest 3 ways of fitting scaling laws - we'll go over each.

The methods (minus third) suggest similar constants. More on this later..

Method 1 - Minimum over runs

Similar to the FLOPS figure on Kaplan - the minimum over the union of all training curves is a power law.

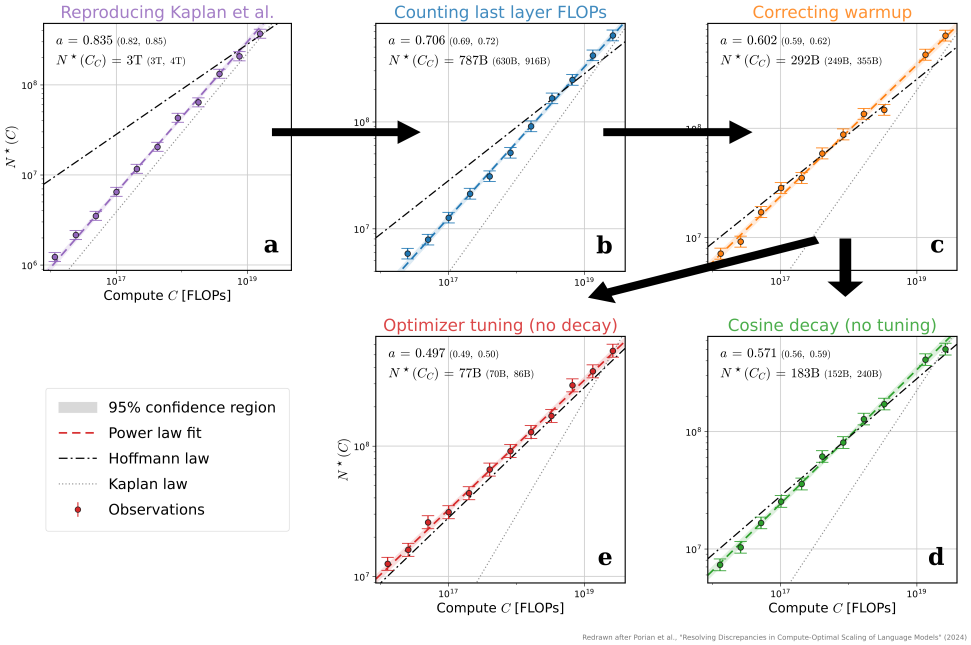
Method 2 - IsoFLOPS

Pick a range of FLOP budgets, vary the total parameter count, take the min over these convex shapes. The minima form a power law.

Method 3 - Joint fits

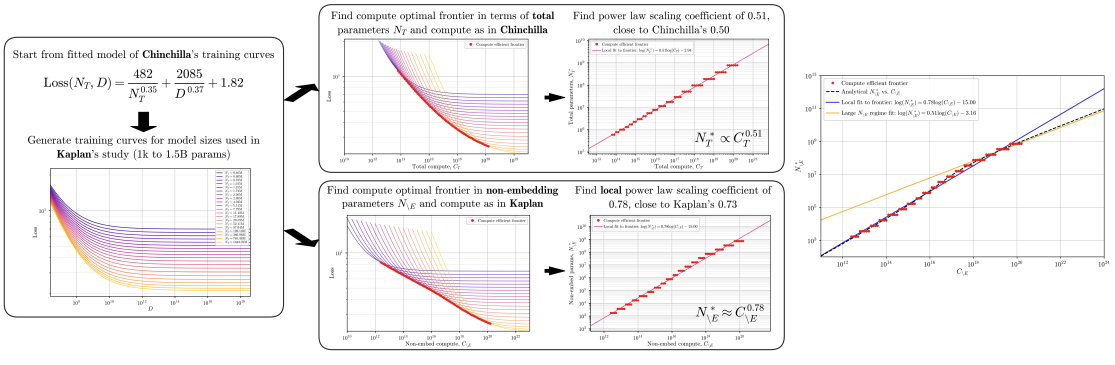
Run a bunch of models on the size-data grid. Use least squares to fit a joint scaling law.

Explanation 1 - Resolving discrepancies in compute-optimal scaling of LMs



- Kaplan removed last layer norm param from the count
- Warmup at very small compute budgets was too high

Explanation 2 - Reconciling Kaplan and Chinchilla scaling laws



Non-embedding vs total param choice + small nonlinearities.

Fun addendum - errors in Chinchilla method 3

Note that this method three was likely flawed in the original paper. Some authors did data forensics, recovered the raw data, and re-did the fit and got results more consistent with methods one and two.

Important note: train-optimal is likely not what you want

Chinchilla aims to tell you what gives the best model for fixed training compute .. But most of the compute in a real deployment is inference .. So we should ”over” train ..

- **GPT3** - 2 tokens / param
- **Chinchilla** - 20 tokens / param
- **LLaMA65B** - 22 tokens / param
- **Llama 2 70B** - 29 tokens / param
- **Mistral 7B** - 110 tokens / param
- **Llama 3 70B** - 215 tokens / param

The more usage we expect, the more it becomes worth it to pay the upfront cost.

IsoFLOPS everywhere

Methods like IsoFLOPS are pretty easy to execute and usually pretty clean.

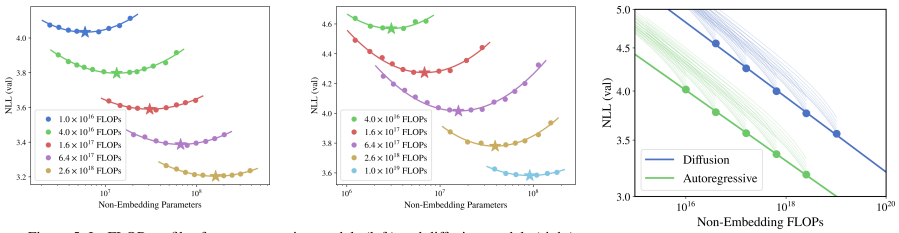
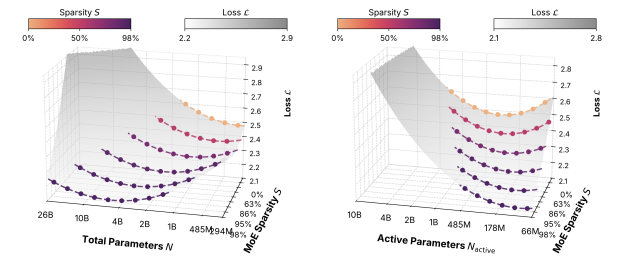


Figure 5: IsoFLOP profiles for autoregressive models (left) and diffusion models (right).

Diffusion - Gulrajani+ 2023.



(a) IsoFLOP surface over sparsity and total parameters

(b) IsoFLOP surface over sparsity and active parameters

MoEs – Abnar+ 2025.

Recapping the scaling - surprising and useful!

- **Data scaling**: understand how data affects models, clean theory
- **Model scaling**: dramatically reduce costs for training
- **Scaling as prediction**: understand what problems can be ”brute-forced”